

Analizzatore DSP: Motore di Analisi Audio

FFT, LUFS, Correlazione, Clipping e Metriche in Tempo Reale

- FFT, LUFS, Correlazione, Clipping e Metriche in Tempo Reale

Categoria: Elaborazione Audio / DSP

Dipendenze: JUCE DSP Module, FFTReal, SIMD (opzionale)

- 1. Panoramica del Motore DSP

Il motore di analisi DSP di WhyCremisi è il sistema sensoriale dell'agente AI — il suo equivalente del sistema uditivo umano. Opera su buffer audio in tempo reale (64–1024 samples) e produce metriche strutturate che alimentano sia la UI che i modelli decisionali dell'AI.

◆ PROCESS FLOW

ANALIZZATORE DSP

Audio In → Ring Buffer → FFT Engine

→ LUFS Peak Hold

→ Phase Correlazione

→ Clipping Spettro

→ Bands Bande

ProcessAggregator

(16ms intervals)

OscBridge

→ JSON msg

Ogni blocco audio processato attraversa l'intera catena di analisi. I risultati sono aggregati ogni 16ms (circa 62 volte al secondo a 48kHz) e inviati al layer di presentazione.

— 2.1 CATENA DI ANALISI FFT

L'analisi spettrale si basa su una FFT con finestra di Hanning:

◆ OVERVIEW

Parametri FFT:

Dimensione finestra 2048 samples
Hop size 512 samples (75% overlap)
Finestra Hanning (von Hann)
Zero-padding No
Frequenza minima 21.5 Hz (a 48kHz)
Frequenza massima 24.0 kHz (Nyquist)
Risoluzione in freq. 23.4 Hz (per bin a 48kHz)

[NOTE] L'overlap del 75% garantisce una risoluzione temporale di ~10.6ms a 48kHz, sufficiente per transienti percettivamente rilevanti mantenendo il carico computazionale sotto controllo.

Implementazione FFT (pseudocodice):

```
void Analyzer::processFFT(const float* channelData, int numSamples) {  
    // Copia nel buffer circolare FFT  
    fftBuffer.write(channelData, numSamples);  
  
    if (fftBuffer.available() ≥ fftSize) {  
        fftBuffer.read(windowBuffer, fftSize);  
  
        // Applica finestra di Hanning  
        for (int i = 0; i < fftSize; ++i)  
            windowBuffer[i] *= hanningWindow[i];  
  
        // Esegui FFT  
        fft.performRealOnlyForwardTransform(windowBuffer);  
  
        // Calcola magnitudine (dB) per ogni bin  
        for (int bin = 0; bin < numBins; ++bin) {  
            float real = windowBuffer[bin * 2];  
            float imag = windowBuffer[bin * 2 + 1];  
            magnitude[bin] = sqrtf(real * real + imag * imag);  
            magnitudeDB[bin] = 20.0f * log10f(magnitude[bin] + 1e-12f);  
        }  
    }  
}
```

— 2.2 BINS DELLO SPETTRO

Con FFT size 2048, il numero di bins utili (fino a Nyquist) è 1024. Ogni bin ha una larghezza di ~23.4Hz a 48kHz.

◆ OVERVIEW

BIN FREQUENZA (Hz) NOTE

0 0 DC component (scartato)

1 23.4 Sub bass estremo

2 46.9 Sub bass

... ...

43 1007.8 Medio-bassi

128 3000.0 Presence

256 6000.0 High-mid

512 12000.0 Brillanza

1024 24000.0 Nyquist (scartato)

— 2.3 CANALI STEREO

L'analisi viene eseguita indipendentemente su entrambi i canali:

```
struct StereoAnalysis {
    float leftMagnitude[1024];
    float rightMagnitude[1024];
    float leftPhase[1024];
    float rightPhase[1024];
    float correlationValue; // [-1.0, 1.0]
    float midMagnitude[1024]; // L+R
    float sideMagnitude[1024]; // L-R
};
```

• 3. Metriche di Loudness

— 3.1 ALGORITMO LUFS (ITU-R BS.1770-4)

WhyCremisi implementa la misurazione di loudness secondo lo standard ITU-R BS.1770-4, con tre finestre temporali:

◆ OVERVIEW

Tipo Finestra Aggiornamento

Momentary (M) 400 ms Ogni 100 ms

Short-term (S) 3 s Ogni 300 ms

Integrated (I) ∞ (cumul.) A ogni blocco

◆ OVERVIEW

Catena di elaborazione LUFS:

Audio In

K-Weighting Filter

Pre-filter HP IIR: s = 38.0 Hz

(shelving)

RLB-weighting HP IIR: s = 168.1 Hz

(high-pass) 2° ordine Butterworth

Mean Square per

channel $(1/N) * \sum x[n]^2$

→ →

Channel 1 Channel 2

Gain: 1.0 Gain: 1.0 (stereo — ITU weights)

Somma pesata

Loudness = $-0.691 + 10 * \log_{10}(\sum G_i * x_i^2)$

+ $10 * \log_{10}(\sum \dots)$

— 3.2 GATING

L'ITU-R BS.1770-4 richiede un gating a due stadi per il calcolo dell'Integrated LUFS:

◆ OVERVIEW

Gating Algorithm:

Stadio 1 — Absolute Gate:

Soglia: -70 LUFS (silenzio)

Blocchi sotto soglia → esclusi dal calcolo

Stadio 2 — Relative Gate:

Calcola loudness media dei blocchi sopra -70 LUFS

Soglia relativa: media + 10 LUFS (tipico: -25 LUFS)

Blocchi sotto soglia relativa → esclusi dal calcolo

Risultato: Integrated LUFS = media dei blocchi rimanenti

[NOTE] Il gating relativo è cruciale: senza di esso, il silenzio tra le tracce abbasserebbe artificialmente il valore di loudness integrato, portando a masterizzazioni non conformi agli standard broadcast.

— 3.3 TRUE PEAK

Il True Peak viene calcolato via oversampling 4x con filtro di ricostruzione sinc:

True Peak = `max(|signal oversampled 4x|)`

Precisione: `±0.1 dB` (conforme a ITU-R BS.1770-4)

Utilizzato per: `limitazione sicura, mastering broadcast`

• 4. Correlazione di Fase

— 4.1 PHASE CORRELATION METER

Il correlation meter misura la similarità tra canale sinistro e destro:

◆ OVERVIEW

CORRELAZIONE STEREO

-1.0 0.0 +0.5 +1.0



→ Out of Mono Corr.

phase compat alto

CREMISI #DC143C (out of phase)

AMBER #FFB000 (mid range)

GREEN #22c55e (in phase)

Formula matematica:

◆ OVERVIEW

$$\text{correlation} = \Sigma(L[n] * R[n]) / \sqrt{(\Sigma(L[n]^2) * \Sigma(R[n]^2))}$$

Valori interpretati:

+1.0 → Identici (mono perfetto)

+0.5 → Buona compatibilità mono

0.0 → Completamente decorrelati

-0.5 → Problemi di fase evidenti

-1.0 → Inversione di fase totale

— 4.2 ANALISI MID/SIDE

◆ OVERVIEW

$$\text{Mid (M)} = L + R \text{ (tutto ciò che è comune)}$$

$$\text{Side (S)} = L - R \text{ (tutto ciò che è differenza)}$$

Rapporto M/S:

M >> S → Mix stretto, centrato

M = S → Mix bilanciato, spazioso

M << S → Mix largo, possibili problemi di fase

— 4.3 VECTORSCOPE / PHASE SCOPE

Il vectorscope è un display L/R a 2D che traccia l'ampiezza istantanea:

◆ OVERVIEW

Canale Sinistro

-1.0 +1.0

Canale Destro

Forme caratteristiche:

Linea 45° → Mono perfetto

Cerchio → Stereo bilanciato

Linea orizz. → Solo sinistro (panned L)

Linea vert. → Fuori fase 180°

• 5. Clipping Detection

— 5.1 PEAK HOLD CON RMS

◆ OVERVIEW

RILEVATORE DI CLIPPING

Soglia configurabile: $[-\infty \dots 0]$ dBFS

Default: -0.5 dBFS

Decay: 0.5 – 10 dB/s

Hold: 0 – 5000 ms

History: ultimi 1024 eventi

Output:

clipDetected: bool (true se clip)

clipCount: int (totale sessione)

peakValue: float (-150 .. 0 dBFS)

holdValue: float (peak tenuto)

lastClipTime: double (samples o ms)

— 5.2 HISTORY BUFFER

```
struct ClipEvent {
    double timestamp;    // campione assoluto
    int    trackIndex;   // ID traccia
    int    channelIndex; // 0=L, 1=R
    float  peakValue;    // valore di picco in dBFS
};

class ClipHistory {
    std::array<ClipEvent, 1024> buffer;
    int writeIndex;
    int count;
    std::atomic<int> totalClips;

    void recordClip(ClipEvent event);
    const ClipEvent* getLastClip() const;
    int  getTotalClipCount() const;
    void autoReset(double timeoutMs);
    void clear();
};
```

— 5.3 AUTO-RESET

◆ OVERVIEW

Modalità di reset:

Manuale → Utente clicca "reset"

Timeout → Dopo N ms senza nuovi clip

Threshold → Clip sotto nuova soglia

Track change → Cambio traccia corrente

• 6. Spettro di Frequenza

— 6.1 SCALA LOGARITMICA 20HZ-20KHZ

L'asse delle frequenze viene mappato in scala logaritmica per corrispondere alla percezione umana:

◆ OVERVIEW

Mappatura pixel → frequenza (schermo 768px):

px freq (Hz) nota

0 20 Sub bass

128 55 A1 (basso)

256 151 D3

384 415 G#4

512 1140 D#6

640 3136 G7

768 20000 Brillanza

Formula: $\text{freq} = 20 * \text{pow}(20000/20, \text{pixel} / \text{larghezza})$

— 6.2 BANDE DI FREQUENZA AGGREGATE

I 1024 bins FFT vengono aggregati in 7 bande musicali per una visualizzazione a barre verticali:

◆ OVERVIEW

Banda Range Bins FFT Colore UI

Sub 20 – 60 Hz 0 – 2 #DC143C (cremisi)

Bass 60 – 250 Hz 2 – 10 #FF6B6B

Low-Mid 250 – 500 Hz 10 – 21 #FFB000 (amber)

Mid 500 – 2 kHz 21 – 85 #FFD700

High-Mid 2 – 6 kHz 85 – 256 #22c55e (green)

Presence 6 – 10 kHz 256 – 426 #00E5FF (cyan AI)

Brilliance 10 – 20 kHz 426 – 853 #00FFaa (verde OK)

```

struct FrequencyBand {
    const char* name;
    float freqMin, freqMax;
    int binStart, binEnd;
    float energy;        // somma magnitudini nel bin range
    float peak;          // picco di energia nel range
    int peakBin;         // bin del picco
    float energyDB;      // energia in dB
};

void Analyzer::computeBands(const float* magnitude,
                           FrequencyBand* bandsOut) {
    for (int b = 0; b < NUM_BANDS; ++b) {
        auto& band = bandsOut[b];
        float sum = 0.0f;
        float maxVal = -INFINITY;
        int maxBin = 0;

        for (int bin = band.binStart; bin < band.binEnd; ++bin) {
            sum += magnitude[bin];
            if (magnitude[bin] > maxVal) {
                maxVal = magnitude[bin];
                maxBin = bin;
            }
        }

        band.energy = sum;
        band.peak = maxVal;
        band.peakBin = maxBin;
        band.energyDB = 20.0f * log10f(sum + 1e-12f);
    }
}

```

— 6.3 PEAK HOLD SPETTRALE

◆ OVERVIEW

Peak Hold:

Decadimento: 2 dB/s (regolabile 0.5–10)

Hold iniziale: 500 ms (nessun decay appena rilevato)

Aggiornamento: a ogni frame FFT

Reset: su richiesta o cambio traccia

Display:

Curva spettrale istantanea (fill)

Curva spettrale peak hold (linea sottile)

[NOTE] Il peak hold con decay lento è essenziale per identificare rapidamente frequenze problematiche: un picco momentaneo viene tenuto in vista abbastanza a lungo da consentire all'AI di suggerire un intervento EQ mirato.

— 7.1 GESTIONE DEI BUFFER

◆ PROCESS FLOW

Gerarchia di buffer:

LIVELLO 1: Buffer di ingresso (JUCE AudioBuffer)

Dimensione: 64–1024 samples (variabile DAW)

Uso: copia immediata → ring buffer

LIVELLO 2: Ring Buffer FFT (lock-free)

Dimensione: 4096 samples (2x FFT size)

Uso: accumulo continuo per finestra FFT

LIVELLO 3: Ring Buffer History (lock-free)

Dimensione: 48000 samples (1s a 48kHz)

Uso: lufficio per analisi ritardate/LUFS

LIVELLO 4: ProcessAggregator (thread-safe)

Dimensione: 64 frames (circa 1s di metrica)

Uso: smoothing temporale, invio OSC/WS

— 7.2 RING BUFFER LOCK-FREE

```
template<typename T, size_t N>
class LockFreeRingBuffer {
    std::array<T, N> buffer;
    std::atomic<size_t> writePos{0};
    std::atomic<size_t> readPos{0};

public:
    bool write(const T* data, size_t count) {
        size_t wp = writePos.load(std::memory_order_relaxed);
        for (size_t i = 0; i < count; ++i) {
            buffer[(wp + i) % N] = data[i];
        }
        writePos.store((wp + count) % N,
                       std::memory_order_release);
        return true;
    }

    bool read(T* out, size_t count) {
        size_t rp = readPos.load(std::memory_order_relaxed);
        // ... implementazione speculare
        return true;
    }

    size_t available() const {
        size_t wp = writePos.load(std::memory_order_acquire);
        size_t rp = readPos.load(std::memory_order_acquire);
        return (wp ≥ rp) ? (wp - rp) : (N - rp + wp);
    }
};
```

— 7.3 CONSIDERAZIONI SIMD

◆ OVERVIEW

Operazioni vettorizzabili (con SIMD/AVX):

- ✓ Moltiplicazione finestra Hanning (vettore × vettore)
- ✓ Magnitudine: $\sqrt{\text{real}^2 + \text{imag}^2}$ per 8 bins
- ✓ Somma LUFS: mean square su blocchi di 8 samples
- ✓ Correlazione: prodotto $L[n] \times R[n]$ vettorizzato

Speedup stimato vs. scalare:

Hanning window ~4×

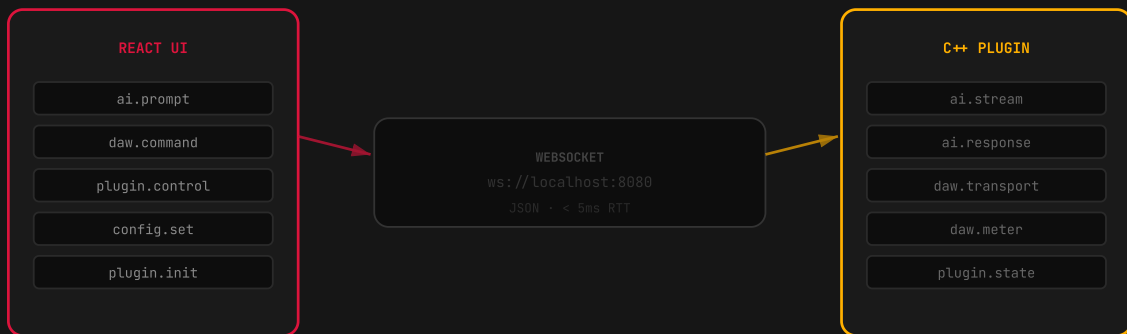
Magnitude compute ~6×

LUFS mean square ~4×

Correlation ~3×

— 7.4 THREAD SAFETY

◆ WEBSOCKET MESSAGE FLOW



[NOTE] La separazione rigorosa tra thread audio e thread UI è fondamentale: il thread audio non può mai essere bloccato da operazioni di I/O o allocazioni di memoria. Tutta la comunicazione avviene tramite strutture dati atomiche lock-free.

— 8.1 CONNESSIONE CON PLUGINPROCESSOR

◆ PROCESS FLOW

PluginProcessor (JUCE)

```
processBlock(audioBuffer, midiMessages)

Analyzer::processBlock(const float** channelData,
int numChannels, int numSamples)

per ogni canale:

processFFT(channel, numSamples)

processLUFS(channel, numSamples)

processCorrelation(channel, numSamples)

processClipping(channel, numSamples)

Analyzer::aggregate()

computeBands(magnitude, bands)

computeMeterData()
```

— 8.2 INVIO VIA OSCBRIDGE

I dati aggregati vengono serializzati in JSON e inviati tramite WebSocket:

```
{
  "type": "daw.analyzer",
  "timestamp": 1715123456789,
  "sampleRate": 48000,
  "metrics": {
    "loudness": {
      "momentary": -14.2,
      "shortTerm": -13.8,
      "integrated": -12.5,
      "truePeak": -1.2
    },
    "correlation": 0.72,
    "clipping": {
      "detected": false,
      "totalClips": 0,
      "peak": -6.3
    },
    "spectrum": {
      "bands": [
        { "name": "sub", "energy": -28.4, "peak": -22.1 },
        { "name": "bass", "energy": -18.2, "peak": -14.5 },
        { "name": "lowMid", "energy": -22.7, "peak": -18.3 },
        { "name": "mid", "energy": -16.5, "peak": -12.2 },
        { "name": "highMid", "energy": -24.1, "peak": -19.8 },
        { "name": "presence", "energy": -30.6, "peak": -26.4 },
        { "name": "brilliance", "energy": -35.2, "peak": -31.0 }
      ]
    }
  }
}
```

— 8.3 FREQUENZA DI AGGIORNAMENTO

◆ OVERVIEW

Metrica Frequenza Latenza max

LUFS Momentary Ogni 100 ms 100 ms

LUFS Short-term Ogni 300 ms 300 ms

LUFS Integrated Ogni blocco N/A (cumul.)

True Peak Ogni blocco 5.3 ms

Correlazione Ogni 16 ms 16 ms

Clipping Ogni 16 ms 5.3 ms (1 hop)

Spettro bande Ogni 32 ms 32 ms

FFT spettro completo Ogni 16 ms 16 ms (full frame)

• 9. Limitazioni e Miglioramenti Futuri

— 9.1 LIMITAZIONI ATTUALI

◆ OVERVIEW

Limitazione Impatto

FFT solo su CPU Elevato carico CPU

con sample rate alti

Mono/Stereo soltanto Nessun supporto

multicanale (5.1, 7.1)

Nessuna separazione fonica Battiti inmiscono

mascheramento spettrale

Finestra Hanning fissa Migliorabile con

finestre adattative

Nessuna compensazione Rumore di fondo

ambientale non filtrato

— 9.2 MIGLIORAMENTI PIANIFICATI

GPU Acceleration per FFT

◆ OVERVIEW

Utilizzo di CUDA/Metal Performance Shaders per

eseguire FFT concorrenti su più canali:

CPU GPU CPU

RingBuf → FFT Batch→ Magnitudo

History 64 frame Aggreg.

Speedup stimato: 10–50× per operazioni FFT batch

Target: GPU integrata (Apple Silicon, Intel Iris)

Machine Learning per Source Separation

◆ PROCESS FLOW

Pre-processing con modello lightweight (Demucs

o Spleeter ridotto) per separare il segnale in

componenti (voce, batteria, basso, altro) prima

dell'analisi:

Audio In → Source Separation → Analyzer per

ogni sorgente

→ EQ suggestion per

sorgente specifica

Espansione Multicanale

◆ OVERVIEW

Supporto futuro per:

5.1 Surround L, R, C, LFE, Ls, Rs

7.1 Surround L, R, C, LFE, Ls, Rs, Rls, Rrs

Ambisonics FFT sferica su ordine 1-3

Oggetti audio Correlazione per oggetto

Miglioramenti Incrementali

- Finestre adattative (Kaiser con beta variabile per compromesso risoluzione/fughe spettrali)
- Zero-padding opzionale per FFT ad alta risoluzione
- Detection automatica del rumore di fondo con soglia adattativa
- Sonogramma in tempo reale con compressione percettiva (mel scale opzionale)
- Cache degli spettri per confronto A/B istantaneo tra stati plugin

1. ITU-R BS.1770-4 — Algorithms to measure audio programme loudness and true-peak audio level (2015)
 2. FFTPack / JUCE DSP Module — Implementazione FFT di riferimento
 3. Zölzer, U. — Digital Audio Signal Processing (3rd ed., Wiley 2022)
 4. Smith, J.O. — Spectral Audio Signal Processing (online, 2011)
-